

Programmation – Récursivité

Récursivité et Arbres Binaires

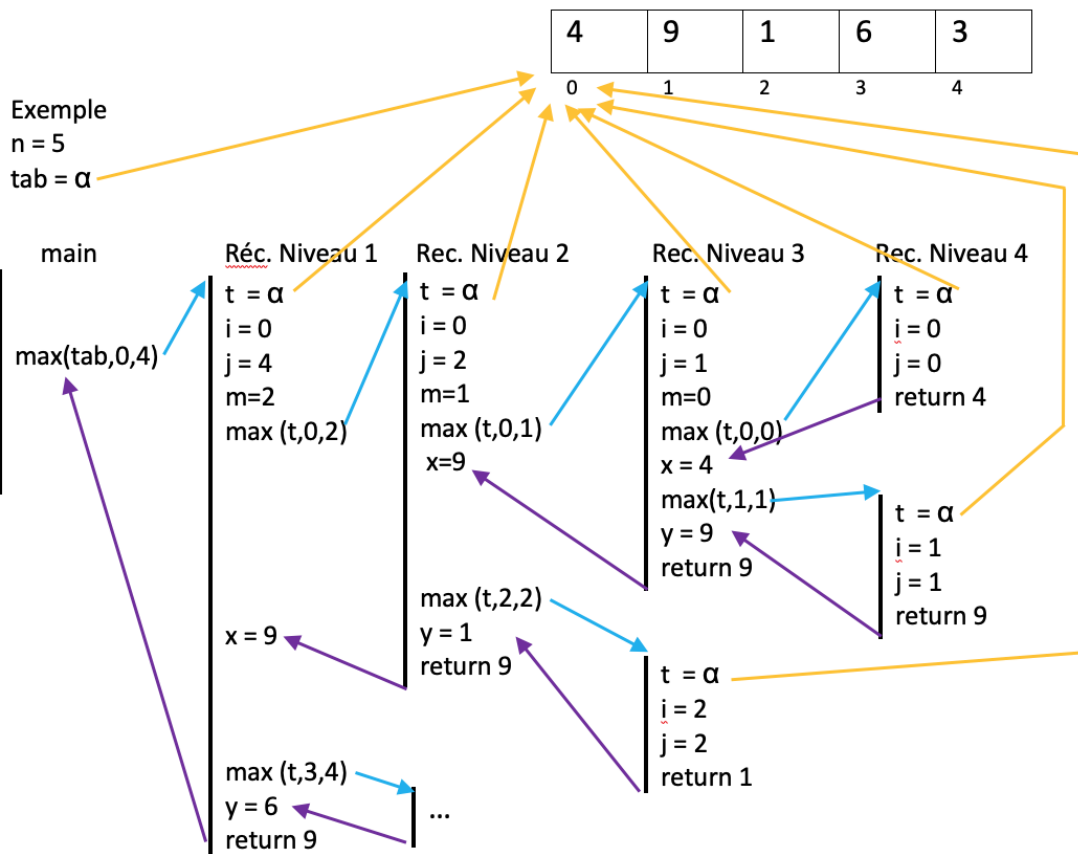
La récursivité est un mécanisme permettant de mettre en œuvre le principe “diviser pour régner” : pour résoudre un problème, on résout le même problème sur des données “plus petites”.

Exemple :

Étant donné un tableau `tab[]` d’entiers, non trié, on veut déterminer la plus grande valeur présente dans `tab`. Une méthode possible (qui n’est pas la plus naturelle) consiste à calculer les moyennes des deux moitiés du tableau et à retenir la plus grande des deux valeurs.

```
1  /**
2   * @pre      i <= j
3   * @return   la plus grande valeur de t[i..j]
4   */
5  public static void max(int[] t, int i, int j) {
6      if (i == j) { return t[i]; }
7      int m = (i + j) / 2;
8      int x = max(t, i, m);
9      int y = max(t, m + 1, j);
10     return (x > y) ? x : y;
11 }
12 public static void main(String[] args) {
13     int n = lireTaille();
14     int[] tab = new int[n];
15     ...
16     System.out.println("le max est " + max(tab, 0, n - 1));
17 }
```

Chaque exemplaire “actif” de la fonction `max` possède ses propres paramètres et variables locales.



Exercice 1

Étant donné un nombre réel r et un nombre entier $n \geq 0$, on veut calculer r^n selon la méthode suivante (on ne veut pas utiliser `Math.pow(r, n)`) :

$$r^0 = 1$$

$$r^k = (r^{k/2})^2, \text{ si } k \text{ est pair } > 0$$

$$r^k = r * (r^{k/2})^2, \text{ si } k \text{ est impair}$$

Exercice 2

Une prairie est peuplée de m moutons, de l loups et de s serpents. L'évolution des espèces est la suivante :

- le matin, chaque loup mange un mouton,
- le midi, chaque serpent mord un loup,
- le soir, chaque mouton écrase un serpent.

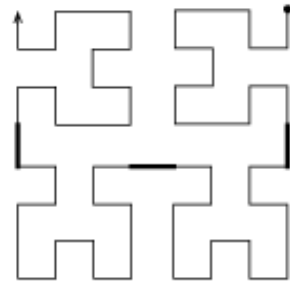
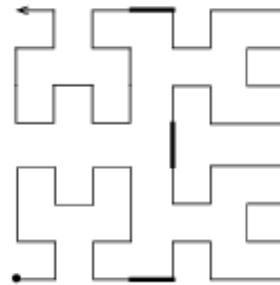
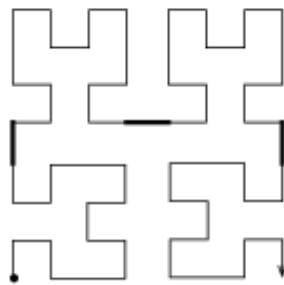
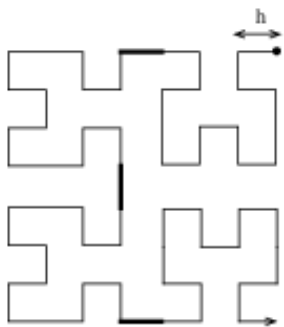
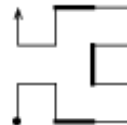
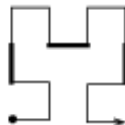
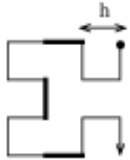
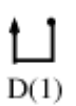
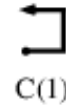
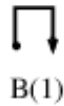
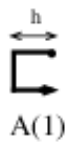
On veut déterminer le nom de la première espèce à s'éteindre.

```
String[] species = {"mouton", "loup", "serpent"};
int m = lireInt(), l = lireInt(), s = lireInt();
syso("l'espèce qui disparaît est :" + species[eliminate(m, l, s, 0)]);

static int eliminate(int n1, int n2, int n3, int period) { ... }
```

Exercice 3

- Étant donnée une fenêtre graphique, on souhaite dessiner, pour $n \geq 0$ et $h \geq 1$ entiers donnés, la figure $A(n)$.
- On dispose de la fonction graphique `trace(u, v)` qui dessine un segment de droite entre le point courant (x, y) et le point $(x + u, y + v)$ (qui devient le nouveau point courant).
- $\forall i > 0$ $A(i), B(i), C(i)$ et $D(i)$ sont constituées par des enchaînements de motifs $A(i - 1), B(i - 1), C(i - 1), D(i - 1)$ reliés par des segments de longueur h .

COURBES DE HILBERT**Remarques :**

- pour $A(i), B(i), C(i)$ et $D(i)$, les segments reliant les motifs $A(i-1), B(i-1), C(i-1), D(i-1)$ ont été mis en évidence par un trait épais,
- un point et une flèche précisent l'origine et la fin du tracé de chaque motif.

Exercice 4

On désire trier par valeurs croissantes un tableau T de n entiers. On suppose disponible :

```

1  /**
2   * @param begin
3   *       position dans le tableau
4   * @param middle
5   *       position dans le tableau
6   * @param end
7   *       position dans le tableau
8   * @param array
9   *       tableau à trier
10  * @pre 0 <= begin <= middle < end <= n-1
11  * @pre les sous-tableaux array[begin..middle] et array[middle+1..end]
12  *       sont triés par valeurs croissantes
13  * @post array[begin..end] est trié par fusion des deux sous-tableaux;
14  *       array[0..begin-1] et array[end+1..n-1] sont inchangés
15  */
16 public static void merge(int begin, int middle, int end, int[] array)
17 */
18 {
19     int[] array1 = new int[middle - begin + 1];
20     int[] array2 = new int[end - middle];
21     /* array1 <- copie de array[begin..middle],
22        array2 <- copie de array[middle+1..end]
23        array[begin..end] <- fusion de array1 et array2
24     */
25 }

```

Écrire

```

1  /**
2   * Trier récursivement array[begin..end] utilisant merge.
3   * N.B. sort(0, n - 1, array) effectue le tri de array par valeurs
4   *       croissantes.
5   *
6   * @param begin
7   *       première position dans le tableau
8   * @param end
9   *       dernière position dans le tableau
10  * @param array
11  *       tableau à trier
12  * @pre i <= j
13  */
14 public static void sort(int begin, int end, int[] array)

```